

Zaszyte błędy danych i jak je naprawić

Dataset celowo zawiera kontrolowane problemy z jakością danych. Zadaniem warstwy Gold jest sprawić, by raport Power BI ich nie „zobaczył”.

6

kontrolowanych błędów + 1 quirk źródła · reguły modulo · w pełni reprodukowalne

Złota zasada: najpierw discovery (sprawdź ziarno i ryzyko), potem buduj Gold. 08_risk_register → świadoma decyzja → zapisz w kontrakcie BI → zwaliduj.

Przegląd: 6 błędów na jednej tabeli

#	BLAD	LOKALIZACJA	REGULA SEEDUJACA	WZORZEC NAPRAWCZY
1	Data z przyszłości	sales_orders.order_date	order_id % 997	flaga is_future_dated
2	Status spoza słownika	sales_orders.status	order_id % 101 → 'Unknown'	CASE WHEN status='Completed'
3	Brak ceny	order_lines.unit_price	line_id % 1543 → NULL	coalesce miar do 0
4	Sierocy klient	order_lines.customer_id	line_id % 4001	Unknown member w dim_customer
5	Sierocy produkt	order_lines.product_id	line_id % 3001	Unknown member w dim_product
6	Rozjazd naglowek/linie	sales_orders.order_total_amount	order_id % 401	Revenue liczony z linii

Osobno — znany quirk źródła: potrojone / zduplikowane linie w order_lines. **Nie jest liczony jako błąd**, bo ziarno fact_sales to line_id (unikalne) i nie da się go zduplikować po kluczu biznesowym. Szczegóły na koncu.

Kto sprząta co — macierz czyszczenia per tabela Gold

Błędy żyją w silver. Każda tabela Gold, budując się z Silver, musi usunąć/obsłużyć błędy, które jej dotyczą — zanim trafi do Power BI.

TABELA GOLD	CZYTA Z	BŁĘDY	CO KONKRETNIE ZROBIĆ
dim_date	sequence()	brak — generowana	tylko zapewnij unikalny date_key
dim_customer	silver.customers + sieroty	#4	Unknown member (LEFT ANTI JOIN); unikalny customer_id
dim_product	silver.products + sieroty	#5	Unknown member; unikalny product_id
fact_sales	silver.order_lines	#1 #2 #3 #6	flagi statusu (is_completed ...); coalesce NULL price/ miar; flaga is_future_dated ; revenue tylko z line_revenue
fact_sales_dashboard	fact_sales + wymiary	#4 #5 #2	COALESCE(..., 'Unknown') po JOIN; flagi statusu
kpi_daily / revenue_monthly	fact_sales	#2 #1	filtr status='Completed' ; pomin is_future_dated

Najwięcej pracy jest w fact_sales — to jedyna tabela czytająca brudne order_lines , więc ładuje tam 4 z 6 błędów. Wymiary obsługują tylko swoje sieroty, a agregaty dziedziczą już czysty fakt.

Szczegóły — błąd po bledzie

1 Data z przyszłości

Gdzie: `silver.sales_orders.order_date` · **Regula:** `order_id % 997 = 0` (data = `current_date()+30`) · Psuje os czasu i trendy.

WYKRYJ

```
SELECT COUNT(*) FROM silver.sales_orders WHERE order_date > current_date();
```

NAPRAW — oznacz flaga (zostaw wiersz)

```
CASE WHEN order_date > current_date() THEN true ELSE false END AS is_future_dated
```

Kontrakt BI: „KPI mogą wykluczać `is_future_dated`”.

2 Status spoza słownika

Gdzie: `silver.sales_orders.status` · **Regula:** `order_id % 101 = 0` → 'Unknown' · **Słownik:** Completed · Returned · Cancelled

WYKRYJ

```
SELECT status, COUNT(*) FROM silver.sales_orders
WHERE status NOT IN ('Completed', 'Returned', 'Cancelled')
GROUP BY status;
```

NAPRAW — flagi statusu, licz miary jawnie po fladze

```
SUM(CASE WHEN is_completed THEN line_revenue ELSE 0 END) AS revenue
```

Nigdy nie licz `SUM(line_revenue)` bez filtra statusu — 'Unknown' wpadnie do wyniku.

3 Brak ceny jednostkowej

Gdzie: `silver.order_lines.unit_price` · **Regula:** `line_id % 1543 = 0` → NULL (przez co `line_revenue/line_margin` też NULL) · Zanizone revenue.

WYKRYJ

```
SELECT COUNT(*) FROM silver.order_lines WHERE unit_price IS NULL;
```

NAPRAW — coalesce miar do 0 (zostaw wiersz)

```
coalesce(unit_price, 0), coalesce(line_revenue, 0), coalesce(line_margin, 0)
```

Nie zostawiaj NULL „w locie”. Gold ma mieć komplet niezerowych miar.

4 Sierocy customer_id (brak w customers)

Gdzie: `silver.order_lines.customer_id` · **Regula:** `line_id % 4001 = 0` · Zrywa relacje do `dim_customer` — przy INNER JOIN wiersze znikaja.

WYKRYJ (LEFT ANTI JOIN)

```
SELECT COUNT(*) FROM silver.order_lines l
LEFT ANTI JOIN silver.customers c ON l.customer_id = c.customer_id;
```

NAPRAW — wzorzec „Unknown member” (Kimball): dorzuc zastępcze wiersze do wymiaru

```
SELECT DISTINCT
  ol.customer_id,
  'Unknown' AS customer_name, 'Unknown' AS city,
  'Unknown' AS customer_region, 'Unknown' AS country,
  'Unknown' AS segment, NULL AS created_date
FROM silver.order_lines ol
LEFT ANTI JOIN silver.customers c ON ol.customer_id = c.customer_id
WHERE ol.customer_id IS NOT NULL
```

Efekt: relacja `fact` → `dim_customer` jest pełna, a osierocone transakcje grupują się pod „Unknown” zamiast znikac.

5 Sierocy product_id (brak w products)

Gdzie: `silver.order_lines.product_id` · **Regula:** `line_id % 3001 = 0` · Zrywa relacje do `dim_product`.

WYKRYJ

```
SELECT COUNT(*) FROM silver.order_lines l
LEFT ANTI JOIN silver.products p ON l.product_id = p.product_id;
```

NAPRAW — ten sam wzorzec „Unknown member” co przy kliencie (pkt 4), tylko dla `dim_product`.

6 Rozjazd: suma nagłowka vs suma linii

Gdzie: `sales_orders.order_total_amount` vs `SUM(order_lines.line_revenue)` · **Regula:** `order_id % 401 = 0` · Dwie różne „prawdy” o wartości zamówienia.

WYKRYJ (rekoncyliacja)

```
WITH line_totals AS (
  SELECT order_id, ROUND(SUM(line_revenue),2) AS line_revenue_total
  FROM silver.order_lines GROUP BY order_id
)
SELECT o.order_id, o.order_total_amount, l.line_revenue_total,
  ROUND(o.order_total_amount - l.line_revenue_total, 2) AS amount_diff
FROM silver.sales_orders o JOIN line_totals l ON o.order_id = l.order_id
WHERE ABS(ROUND(o.order_total_amount - l.line_revenue_total, 2)) > 0.01
ORDER BY ABS(amount_diff) DESC;
```

NAPRAW — wybierz jedno źródło prawdy

Revenue zawsze z linii (`line_revenue`), nigdy z nagłówka. `order_total_amount` traktuj jako kontrolę rekonyliacyjną.

! Quirk źródła (NIE błąd) — potrojone / zduplikowane linie

Gdzie: `silver.order_lines` · Regula: `line_id % 10007 = 0` (re-insert z `line_id + 10000000`)

DLACZEGO TO NIE JEST JEDEN Z 6 BŁĘDÓW

Generator z założenia tworzy **3 identyczne linie na każde zamówienie** (ten sam `product_id`, ta sama `quantity`). Czyli `(order_id, product_id)` występuje **3x naturalnie** — **nie jest kluczem**. `COUNT(*) - COUNT(DISTINCT (order_id, product_id)) = ~240 000` (naturalne); zaszyty re-insert to ~36 wierszy, nieodróżnialnych od potrojenia. Brak zapytania wykrywającego w Dniu 1.

DECYZJA MODELOWA — ziarno = `line_id` (unikalne), więc to poprawna linia

Kontrakt BI: „`fact_sales` ma ziarno `line_id`; źródłowe potrojenie linii i zaszyty re-insert są znane i pozostają w danych”. **Nie deduplikujemy** — dedup po `(order_id, product_id)` obciąłby revenue do ~1/3.